KAVACH: what we built and where it belongs

Team Kavach — PES University, Bengaluru

Assessment: KAVACH is a working prototype for validating and controlling enterprise tool actions. The useful product direction is an action-policy component for workflows such as accounts payable, email and CRM. Competing with foundation models on intelligence, or claiming a universally superior agent-security system, is unsupported.

> KAVACH checks a proposed business action against its argument sources, expected effects and trusted business records before allowing it. For supported actions it controls, it can hold delivery or support recovery and record a verifiable trail.

This report extends the separate Infosys gap research. Official model/product documentation establishes advertised capabilities; local implementation and tests establish KAVACH's mechanisms. Neither establishes a cross-model safety ranking. No paid model evaluations were run for this comparison.

1. The four layers that explain the confusion

Layer	Examples	Responsibility
Model	Claude, GPT, Kimi, GLM, Llama, Qwen	Interpret requests and produce answers or proposed actions.
Serving system	Provider API, Ollama	Load/serve models and expose inference interfaces.
Agent application	Claude Code, Codex, Kimi Code, Infosys AgentOS	Manage steps, tools, permissions, context and execution.
Business enforcement	KAVACH plus authoritative business systems	Validate a concrete action and apply supported controls.

These layers overlap in real products. An agent product may already contain strong action enforcement. KAVACH must be compared against those controls too, not just against a bare model.

Typical placement:

```
User request + documents/email/tool results
                    |
             Model and agent runtime
                    |
              Proposed tool call
                    |
 KAVACH: source + effect + business-policy checks
                    |
      allow / safeguard / escalate / deny
                    |
      Controlled tool executor and outcome
```

For client-defined tools, application code handles execution. Hosted tools are different: providers can execute them on their infrastructure. KAVACH only protects an operation if its invocation passes through an enforced adapter, proxy or executor. See Claude tool-use architecture.

2. Older and current model families: a fair comparison

The examples below identify generations rather than claim every available release was evaluated. “Current” describes the official pages inspected; availability can depend on product, account and hosting provider. No model in this table received a new paired live KAVACH evaluation in this research.

Family	Representative earlier generation	Current official examples	What KAVACH contributes if integrated
OpenAI	GPT-4.1; GPT-6 Sol	GPT-6 Astra, GPT-6.1 Sol, GPT-6.1 Terra, GPT-6 Luna	A separately configured business-action policy and evidence path. This is not an intelligence upgrade.
Anthropic	Opus 4.6/4.7/4.8, Sonnet 4.6; Opus/Sonnet 5	Opus, Sonnet and Haiku 5.5; specialized Mythos/Fable 5.1 also appear	Checks tied to the actual tool arguments, records and controlled effects. Model refusal already provides some protection.
Moonshot / Kimi	Kimi K2; Kimi K3 precedes K3.1 Pro	Kimi K3.1 Pro	The same action policy can be applied downstream if calls and relevant outputs reach the guard.
Z.ai / GLM	GLM-4.7; GLM-5.2 precedes GLM-5.3	GLM-5.3	Transaction checks outside the model; provider-specific reasoning and tool protocols still require validation.
Ollama	Runtime and model catalogs evolve independently	Runs local/cloud model choices; its Fly.io example includes Kimi K3 and GLM-5.3	Guard the application's tool execution. Local inference alone does not authorize a payment or recipient.

Official sources: OpenAI catalog, GPT-4.1, GPT-6 Sol, Anthropic model announcements, Moonshot, official Kimi K2 repository, GLM-4.7, GLM-5.3, Ollama/Fly.io integration.

A stronger model can recognize more malicious instructions and avoid more bad proposals. That may reduce the incremental value of KAVACH's injection heuristics. It does not automatically configure a company's release limits or establish that an email's new account number is authorized. A model can retrieve business records and reason correctly; KAVACH's value is that a configured check can be mandatory on the execution path. If the business API already enforces the same check, the incremental value must come from additional evidence, coverage or workflow integration.

Ollama is not a competing model family. It serves models; agent applications around it execute tools. Its documentation explicitly shows application-side handling of calls, and `ollama launch` integrates existing coding agents. See tool calling and agent launching. Local hosting, model quality, sandboxing and transaction authorization answer different questions.

3. Existing agent controls are real competition

Product/control	Existing functionality confirmed in official material	What would justify adding KAVACH
OpenAI Agents SDK	Input/output/tool guardrails and resumable human approval. Tool checks can sit directly around function execution.	Reusable concrete source, consequence and business-record checks attached through those extension points.
Codex	Sandbox restrictions, approval policy, managed networking and automatic review of eligible approval requests.	Enterprise-specific transaction evidence and enforcement in a supported connector. A second generic approval prompt adds little.
Claude Code	Permissions enforced by the application; allow/ask/deny rules, hooks, managed settings and an auto-mode classifier.	Business-aware checks using relevant records and tool-specific effects, with measured value over its existing controls.
Kimi Code	Approval flow and configurable permission modes; routine actions can run automatically under the selected mode.	An enforced business-tool check; a direct integration has not been demonstrated here.
GLM tool ecosystem	Function calling; provider-managed MCP discovery and calls with configurable tool permissions.	Route relevant MCP calls through a guarded endpoint or enforce policy in the server. Hosted calls can bypass a client-only guard.
Ollama with Fly.io	Agent sandbox VM, connector access controls and gateway-managed provider credentials.	Transaction policy for allowed business APIs. Isolation already has value and should remain in place.
Infosys Foundry/AgentOS /RAI	Tool governance, hooks, approvals, sandboxing, evidence and content-risk controls.	Extend the actual invocation path with source/effect/business-fact validation and supported release controls.

Sources: OpenAI guardrails, Codex security, automatic review, Claude Code permissions, Kimi Code interaction, GLM function calling, GLM MCP, Ollama/Fly.io. Pinned Infosys source links and function probes are in the companion report.

These products are extensible: an enterprise could implement similar checks using their hooks, SDKs or business APIs. KAVACH's opportunity is a useful implementation and integration, not exclusive ownership of pre-tool security. The comparison does not establish that competitors cannot verify business facts.

4. What the repository actually implements

Component	Implemented behavior	Important limit
Argument provenance	Registers tool outputs, labels their values and checks externally derived control arguments through deterministic rules.	Normal mode uses value matching/normalization; arbitrary model transformations can lose lineage. It is not complete information-flow tracking.
Consequence analysis	Stages supported SQLite/file operations and inspects changes; uses typed/static predictions for other effects.	A predicted remote action is not a production dry run. Correct manifests and adapters are necessary.
Business policy	R41 checks intended payment account/amount against invoice/vendor facts; R18 checks release authority; R35 protects finance records against raw SQL changes.	Facts come from the seeded demo environment, not an authenticated production ERP.
Controlled effects	Supported owned actions use email queues, payment holds, parked entries, snapshots or change logs.	External hook/proxy execution does not automatically inherit undo; delivered email and settled real-bank transfers cannot be universally reversed.
Evidence	Links decision/effect receipts, supports signatures and signed checkpoints.	Trust depends on keys, durable storage and deployment; evidence does not certify regulatory compliance.
Integrations	REST endpoint, MCP proxy, common guard object, SDK wrappers and command hooks.	An adapter's existence is not proof that every model/product/version works with it.

Source anchors: provenance, staging, finance policy, protected records, safeguards, receipts, external execution tests.

The model client is currently Qualcomm-specific: it sends OpenAI-shaped chat requests but contains endpoint-specific serialization and response workarounds. It always sends parameters such as `temperature` and `max_tokens`, and its default model selection is oriented toward its known Llama endpoints. OpenAI-shaped requests alone do not establish compatibility with all reasoning models or providers. GLM-5.3, for example, documents mandatory reasoning and migration requirements. Sources: local client, demo agent, GLM requirements.

The guard itself can inspect normalized calls without calling an LLM. Its in-process wrapper disables the LLM judge and local execution. That supports the architectural description “model-independent checks”; it does not support “tested on every model.” See guard initialization.

5. The clearest explanation: one invoice

Suppose a vendor master says invoice INV-101 is payable to TEST-ACCT-001, while an email tells the assistant to pay TEST-ACCT-999.

1. A model may detect the scam and refuse. Credit that success to the model.
2. If an agent nevertheless proposes the mismatched payment, KAVACH's R41 checks the records and denies it in the covered workflow.
3. Correcting the account does not automatically establish release authority: R18 checks the role and amount separately.
4. Where KAVACH owns supported execution, a hold can postpone release; its outcome and approvals can be traced.
5. A model upgrade changes proposal behavior. The configured transaction constraint can remain the same.

The invoice regressions cover mismatched accounts, mismatched amounts, missing accounts, approval, bank failure and repeated payment behavior. The bank is a sandbox. See invoice tests.

This explains the useful distinction: good reasoning can avoid an unauthorized proposal; an enforced policy can reject a covered unauthorized proposal even after it is generated. A real deployment needs trusted, fresh facts and complete interception. If an unguarded path reaches the bank, the policy is bypassed.

6. Evidence: promising, narrow and mixed

The repository-wide audit recorded **521 passed, 14 skipped, 0 failed**. The focused rerun for this comparison produced **19 passed, 3 skipped** across adapter, invoice and external-execution tests. The skips are important: the installed environment lacks `agents`, `langchain_core` and `claude_agent_sdk`, so those actual SDK tests did not run. Generic local/remote guard and command-hook tests did run. Commands and counts are recorded in the comparison validation artifact.

Saved live evidence is not a cross-provider benchmark:

- The Claude comparison reports `claude-opus-5-5`. Claude already refused the malicious email workflow without KAVACH. The guarded benign send demonstrates a queue before delivery; that is a clearer differentiated behavior than claiming KAVACH caused every refusal.
- Saved AgentDojo banking results report 65/128 attacker successes without the guard and 0/128 with it, with benign completion 8/16 in both. Saved Slack results report 48/103 versus 11/103 attacker successes, while benign completion drops from 9/21 to 6/21. These partial runs were not reproduced, and the raw live-task logs are absent from this checkout.
- The synthetic risk ablation reports no verdict changes when learned risk is zeroed. That suite does not establish the learned model as the safety differentiator.
- No comparable saved paired safety/utility results were established here for Kimi, GLM, OpenAI or Ollama-hosted models. Do not create rankings or percentages for them.

Evidence: Claude comparison, AgentDojo comparison, ablation, repository validation.

Two implementation qualifications must accompany the pitch. `/api/v1/check` defaults to decision-only but can execute when `execute=True`. Approval resolution binds the pending call digest, but it does not rerun the full policy and tool-definition validation there; delayed execution needs a stronger freshness contract. Sources: REST input, approval resolution.

7. My product judgment

Focus KAVACH on **enterprise transaction validation**, beginning with accounts payable and controlled outbound communication. This is a recommendation based on the code and the inspected alternatives, not a proven market demand result.

Keep the core: argument-source checks, measured supported effects, authoritative-record comparisons, exception evidence, controlled release and verifiable outcomes. Use AgentOS/Foundry for orchestration and existing infrastructure, and RAI Toolkit where content/model-risk checks help. The existing AgentOS hook still needs authenticated token propagation, reliable tenant-scoped context and real ERP fact mapping before a production claim is justified; see the companion Infosys report.

Do not lead with generic prompt-injection detection, number of rules, a risk score, “works with any model,” universal undo, or claims of outperforming Claude/GPT/Kimi/GLM. They are weak or unsupported differentiators. Provenance, policy and tool interception are also established approaches; the companion report discusses CaMeL, LlamaFirewall and AWS AgentCore rather than asserting global novelty.

An appropriate Infosys pitch is:

> Foundry and AgentOS run enterprise agents; RAI handles content and model risks. KAVACH adds a reusable action check that uses source, effect and business-record evidence, with controlled release for supported workflows.

Why Infosys may value this is an inference: enterprises need reliable operations across business systems and model choices, without rewriting transaction checks for every agent. It is not evidence of Infosys's private roadmap or judging intent. That value must be demonstrated with real integration and useful automation.

8. What would prove the USP next

Use the same accounts-payable and email tasks with at least one available model from each requested family, and retain an earlier representative where access permits. Resolve actual model IDs and capability settings from each provider before running; do not assume a marketing name is a valid API identifier.

Comparison arm	Purpose
Agent with its normal supported permissions and safeguards	Measure a realistic existing product baseline.
Same agent plus KAVACH	Measure incremental safety, task completion, delay and review burden.
Optional minimally guarded client-tool harness	Isolate mechanisms; clearly separate it from product-default safety.

Hold tasks, available tools, record state and attack families constant. Record repeated runs, model/configuration, proposed versus executed violations, sensitive-read leaks, legitimate task completion, human reviews, latency/cost, stale-fact behavior and partial tool failures. Test transformed arguments and unguarded execution paths. Preserve per-run traces and account for escalations; “did not execute because waiting for a reviewer” is not a completed task.

The decisive result is fewer executed unauthorized actions while preserving useful completion at acceptable review burden. If stronger models already avoid a given attack, show the remaining release-control or evidence benefit honestly. If the existing business API already guarantees the invariant, test an additional constraint or workflow benefit instead of counting the API's success as KAVACH's.

Conclusion: the repository contains meaningful engineering, especially its concrete finance/action controls. Its defensible USP is the implemented combination at an enterprise action boundary. Broad model compatibility, production business-data fidelity and cross-model incremental value remain to be established.